



MINISTÉRIO DA EDUCAÇÃO
SECRETARIA DE EDUCAÇÃO PROFISSIONAL E TECNOLÓGICA
INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
BAIANO - CAMPUS GUANAMBI

MARIA CLARA MACIEL DA SILVA
JÉSSICA TAINÁ RODRIGUES SILVA
TAINARA DO AMARAL OLIVEIRA AZEVEDO

APLICATIVO MOBILE DE GERENCIAMENTO DE SERVIÇOS:
CoIISERVICE

Trabalho apresentado como
requisitos para aprovação na
disciplina de programação para
dispositivos móveis e sem fio do
curso de Tecnologia em Análise e
Desenvolvimento de Sistemas do
Instituto Federal de Educação,
Ciência e Tecnologia Baiano –
Campus Guanambi.

Docente: Wellison Brito

Guanambi – BA

2026

Visão Geral do Produto

Esse aplicativo será desenvolvido em Flutter voltado ao gerenciamento completo de ordens de serviço (OS) para empresas especializadas em climatização e manutenção de eletrodomésticos, incluindo ar-condicionado, geladeira, freezer e micro-ondas.

Problema	Gestão manual de chamados gera perda de informação, atrasos no faturamento e dificuldade de rastreo de pagamentos.
-----------------	--

Solução	Plataforma centralizada onde clientes são cadastrados, serviços são agendados com formulários pré-definidos ou dinâmicos, técnicos são alocados e o status de pagamento é monitorado com alertas automáticos.
----------------	---

Público-alvo	Empresas de climatização e eletrodomésticos com equipes de campo (técnicos) e atendimento ao cliente.
---------------------	---

Plataforma	Mobile (Flutter) — Android — com backend REST
-------------------	---

REQUISITOS FUNCIONAIS (RF)

- RF-01 — Autenticação e Controle de Acesso
- RF-01.1 — O sistema deve autenticar usuários via usuario e senha.

RF-01.2 — O sistema deve suportar os perfis: Administrador e Funcionário, com permissões distintas.

RF-01.4 — O administrador deve poder ativar e desativar contas sem excluir o

histórico do funcionário.

RF-02 — Cadastro de Clientes

RF-02.1 — O sistema deve permitir cadastrar clientes com: nome completo, CPF/CNPJ, endereço, telefone e e-mail.

RF-02.2 — O sistema deve exibir o histórico de OS e situação de pagamento no perfil do cliente.

RF-02.3 — O sistema deve permitir a busca de clientes por nome.

RF-02.4 — O sistema deve permitir editar dados de clientes já cadastrados.

RF-03 — Cadastro de Funcionários

RF-03.1 — O administrador deve poder cadastrar funcionários com: nome, CPF, especialidade e contato.

RF-03.2 — O sistema deve exibir a lista de funcionários.

RF-03.3 — O administrador deve poder desativar um funcionário sem excluir seu histórico de OS.

RF-04 — Catálogo de Serviços

RF-04.1 — O administrador deve poder cadastrar serviços com nome, descrição e preço fixo.

RF-04.2 — Cada serviço deve ter um formulário de coleta de dados pré-definido associado.

RF-04.3 — O sistema deve exibir o menu de serviços disponíveis para os atendentes.

RF-04.4 — O administrador deve poder ativar e desativar serviços do catálogo.

RF-05 — Abertura e Gestão de Ordens de Serviço (OS)

RF-05.1 — O Funcionário deve poder abrir uma OS vinculando um cliente e selecionando o serviço desejado.

RF-05.2 — Ao selecionar um serviço com formulário associado, o sistema deve pré-carregar o formulário automaticamente.

RF-05.3 — Para chamados de revisão diagnóstica (cliente não sabe o problema), a OS deve ser aberta sem formulário pré-definido, a ser preenchida pelo técnico durante o atendimento.

RF-05.4 — O funcionário deve poder indicar se o serviço é externo (fora do estabelecimento) e informar a distância em km.

RF-05.5 — O sistema deve calcular automaticamente o valor total da OS (preço do serviço + taxa de km quando aplicável).

RF-05.6 — O funcionário deve poder alocar um técnico disponível ao criar a OS.

RF-06 — Acompanhamento de Status

RF-06.1 — A tela inicial deve exibir todas as OS do dia com etiquetas visuais de status: Aberto, Em Andamento, Concluído, Pagamento Pendente e Cancelado.

RF-06.2 — O funcionário deve visualizar apenas as OS atribuídas a ele.

RF-06.3 — O funcionário deve poder atualizar o status da OS durante o atendimento.

RF-06.4 — O funcionário deve poder filtrar OS por status, técnico, tipo de serviço e período.

RF-06.5 — O funcionário deve poder adicionar observações à OS durante o atendimento.

RF-07 — Pagamentos e Cobranças

RF-07.1 — O sistema deve controlar o status de pagamento de cada OS: Pago ou Em Aberto.

RF-07.2 — O funcionário deve poder registrar o pagamento e a forma de pagamento de uma OS.

RF-07.3 — O sistema deve disparar um alerta após X dias aos pagamentos em aberto.

RF-07.4 — O administrador deve poder configurar o prazo (X dias) do alerta de cobrança.

RF-07.5 — O sistema deve disponibilizar relatório de OS com pagamento pendente.

RF-08 — Notificações

RF-08.1 — O funcionário deve ser notificado ao ser alocado a uma OS.

Arquitetura Adotada: MVVM

A escolha do MVVM baseia-se na necessidade de gerenciar formulários complexos, múltiplos estados reativos e regras de negócio de forma escalável.

Principais motivos que justificam essa escolha arquitetural para o seu escopo:

Gerenciamento de Formulários Dinâmicos (Crucial para o App) O sistema exige que a abertura de OS (Ordem de Serviço) seja dinâmica: alguns serviços têm formulários pré-carregados (RF-05.2) e outros são "em branco" para revisão diagnóstica (RF-05.3).

- **Por que MVVM?** O *ViewModel* gerencia o estado de cada campo de forma isolada da interface.

Reatividade e Sincronização em Tempo Real O app lida com mudanças de status constantes (OS "Em Andamento" para "Concluído", alocação de técnicos, pagamentos pendentes).

- **Por que MVVM?** Ele foi projetado para *binding* reativo. No ecossistema Flutter, ele se integra nativamente com ferramentas como Provider, Riverpod ou ValueNotifier. Quando o *ViewModel* recebe uma atualização do Firebase (ex: uma nova OS alocada [RF-08.1]), ele simplesmente avisa a *View*, que se reconstrói automaticamente. O MVC tradicional não possui esse mecanismo nativo de reatividade.

Testabilidade das Regras de Negócio O projeto possui regras financeiras e matemáticas importantes, como calcular o valor total da OS somando o preço fixo com a taxa de quilometragem (RF-05.5) e disparar alertas de cobrança após X dias (RF-07.3).

- **Por que MVVM?** O *ViewModel* é uma classe Dart pura, sem nenhuma dependência visual (widgets). Isso significa que você consegue escrever testes unitários para garantir que o cálculo do GPS ou as regras de cobrança funcionem 100% sem precisar emular a tela do celular.

Isolamento da Origem dos Dados (Offline/Online) O cronograma prevê armazenamento local nas primeiras Sprints (SQLite/SharedPreferences) e sincronização em nuvem (Firebase) na Sprint 4.

Por que MVVM? Ao combinar o MVVM com o *Repository Pattern*, a sua tela (*View*) não faz ideia de onde os dados vêm. A *View* pede os clientes ao *ClientViewModel*; este pede ao *ClientRepository*. Se o celular estiver sem internet, o repositório busca no SQLite; se tiver internet, busca no Firebase. A arquitetura suporta essa transição de tecnologias sem você precisar reescrever a interface.

Alinhamento com o Padrão da Comunidade Flutter O MVVM (e suas variações focadas em estado) é a abordagem mais recomendada pela documentação oficial de arquitetura do Flutter. Isso facilita encontrar soluções para bugs, utilizar bibliotecas de terceiros e manter o código manutenível à medida que as Sprints avançam e o projeto cresce.

Em resumo: O MVVM foi escolhido porque ele lida melhor com o estado dinâmico da tela de forma reativa, permitindo que a interface seja "burra" (apenas exibe dados e capta toques) enquanto o *ViewModel* assume toda a inteligência e preparação dos dados, garantindo um aplicativo robusto e fácil de testar.

Product Backlog

Sprint 2 : Arquitetura e Modelagem

Foco: Definição da base do projeto e padrões de comunicação.

Tecnologia Obrigatória: MVVM, Clean Architecture, Repository Pattern.

ID	Requisito	Descrição Técnica	SP	Requisitos Vinculados (RFs)
1.1	Arquitetura	Estruturação de pastas lib/ seguindo o padrão Feature-First .	3	Todos (Base do Projeto)
1.2	Modelagem	Criação das Entidades Dart puras (Client , Employee , Order).	5	RF-02, RF-03, RF-05
1.3	Contratos	Definição das interfaces abstratas para os Repositórios.	3	Todos (Interface de Dados)

Sprint 3: Organização e Persistência Local

Foco: Armazenamento local e configurações de sistema.

Tecnologia Obrigatória: Feature folders, SharedPreferences, SQLite.

ID	Requisito	Descrição Técnica	SP	Requisitos Vinculados (RFs)

2.1	Configurações	SharedPreferences para salvar o prazo de alerta de cobrança.	2	RF-07.4
2.2	Clientes	CRUD de Clientes (cadastro, busca e edição) via SQLite .	5	RF-02.1, RF-02.3, RF-02.4
2.3	Funcionários	Gestão de técnicos e especialidades com persistência em SQLite .	5	RF-03.1, RF-03.2, RF-03.3
2.4	Catálogo	Cadastro de serviços e associação de formulários via SQLite .	8	RF-04.1, RF-04.2, RF-04.4

Sprint 4: Temas e Layout Responsivo

Foco: Identidade visual, reatividade e adaptação de telas.

Tecnologia Obrigatória: ThemeData, Modo Claro/Escuro, LayoutBuilder.

ID	Requisito	Descrição Técnica	SP	Requisitos Vinculados (RFs)
3.1	Design System	ThemeData global (cores/fontes) e suporte a Dark Mode .	3	Interface Global
3.2	Dashboard	Tela inicial com etiquetas visuais de status via LayoutBuilder .	8	RF-06.1, RF-06.2

3.3	Menu	Interface do catálogo de serviços adaptável para tablets/mobile.	3	RF-04.3
-----	-------------	--	---	----------------

Sprint 5: Serviços e Nuvem

Foco: Integração com Backend, sincronização e regras de negócio.

Tecnologia Obrigatória: Provider, Firebase, Teste Unitário (Mockito).

ID	Requisito	Descrição Técnica	SP	Requisitos Vinculados (RFs)
4.1	Autenticação	Firestore com distinção de perfis (Admin e Técnico).	5	RF-01.1, RF-01.2, RF-01.4
4.2	Ordens (OS)	Lógica de abertura de OS e persistência no Cloud Firestore .	8	RF-05.1, RF-05.2, RF-05.3, RF-05.6
4.3	Estado Global	Gestão de estados das OS e pagamentos utilizando Provider .	5	RF-06.3, RF-07.1, RF-07.2
4.4	Qualidade	Testes Unitários com Mocks para validar o cálculo de valores.	5	RF-05.5

Sprint 6: Sensores e Hardware

Foco: Integração com recursos físicos e notificações.

Tecnologia Obrigatória: GPS (Geolocator), Sensors (FCM), Permissões.

ID	Requisito	Descrição Técnica	SP	Requisitos Vinculados (RFs)
5.1	Localização	Sensor de GPS para cálculo automático da taxa por KM.	8	RF-05.4, RF-05.5
5.2	Notificações	Alertas Push (FCM) para novas alocações de OS aos técnicos.	5	RF-08.1
5.3	Segurança	Fluxo de tratamento de Permissões de hardware no sistema.	3	Sensores

Sprint 7: Acessibilidade

Foco: Inclusão, refinamento final e conformidade.

Tecnologia Obrigatória: Semantics, TalkBack, Fonte Dinâmica.

ID	Requisito	Descrição Técnica	SP	Requisitos Vinculados (RFs)
6.1	Navegação	Implementação de Semantics no histórico e formulários.	5	RF-02.2

6.2	Alertas	Sistema de cobrança com feedback acessível e fontes dinâmicas.	5	RF-07.3
6.3	Auditoria	Relatório final de acessibilidade e documentação da PR final.	3	Qualidade

Responsabilidades por Camada

Camada / Componente	Tecnologia	Exemplos no CoolService
UI Layer — View	Widgets Flutter (Stateless/Stateful)	LoginScreen , ClientProfileScreen (RF-02.2), OrderFormScreen (RF-05.2), ServiceMenuScreen.
UI Layer — ViewModel	ChangeNotifier / Riverpod Notifier	AuthViewModel (RF-01), OrderViewModel (RF-05, RF-06), PaymentViewModel (RF-07), EmployeeViewModel.
Domain — Model / Entity	Classes Dart puras	Client , Employee , ServiceOrder, Payment, ServiceCatalog.
Domain — UseCase	Classes Dart (Lógica de Negócio)	CalcKmFeeUseCase (RF-05.5), UpdateOrderStatusUseCase (RF-06.3), ChargeAlertUseCase (RF-07.3).
Data Repository	abstract class Dart	IOrderRepository , IClientRepository, IAuthRepository.

(Interface)		
Data Repository (Impl.)	— Implementação concreta	FirestoreOrderRepository, FirestoreClientRepository.
Data DataSource	— Firebase Firestore / FCM / API	FirestoreFirestoreDataSource, FCMNotificationDataSource (RF-08.1).

Estrutura de Pastas do Projeto

A estrutura segue o padrão feature-first dentro das camadas definidas pela arquitetura MVVM, facilitando o desenvolvimento paralelo em equipe:

```
lib/
├─ core/
│  ├─ database/
│  │  └─ database_helper.dart
│  ├─ app_config/      <-- O SharedPreferences
│  │  └─ data/
│  │     └─ preferences_service.dart
│  │  └─ presentation/
│  │     └─ viewmodels/
│  │        └─ app_config_view_model.dart
│  └─ widgets/
│     └─ menu_lateral.dart
│
├─ features/
│  ├─ auth/
│  │  └─ data/
│  │  └─ domain/
│  │     └─ presentation/
│  │
│  └─ clientes/
│     └─ data/
│        └─ models/cliente_model.dart
│        └─ repositories/sqlite_cliente_repository.dart
│        └─ domain/
│           └─ entidades/cliente.dart
│           └─ repositories/i_cliente_repository.dart
```

```
| | └ presentation/
| |   └ view_model/cliente_view_model.dart
| |   └ view/cliente_list_page.dart
| |
| └ funcionarios/
|   └ data/...
|   └ domain/...
|   └ presentation/...
|
| └ ordens_servico/
|   └ data/...
|   └ domain/
|     └ presentation/...
```